Fleet management web app

Submitted to: Dr Abdelrhman Abotaleb & Dr Ahmed Hussein

Done by: Mohamed Ahmed Abdelazim Mohamed

Deploy Fleet Management Web APP on AWS EC2

Launch an EC2 Instance

1. Go to EC2 Dashboard.
2. Click Launch Instance.
3. Choose an Amazon Machine Image (AMI):
    - Select Ubuntu 22.04 LTS.
4. Choose an Instance Type:
    - t2.micro(Free Tier).
5. Configure Security Group:
    - Allow Inbound Rules for:
        - SSH (22)
        - HTTP (80)
        - Custom TCP Rule (8501) → Required for Streamlit.

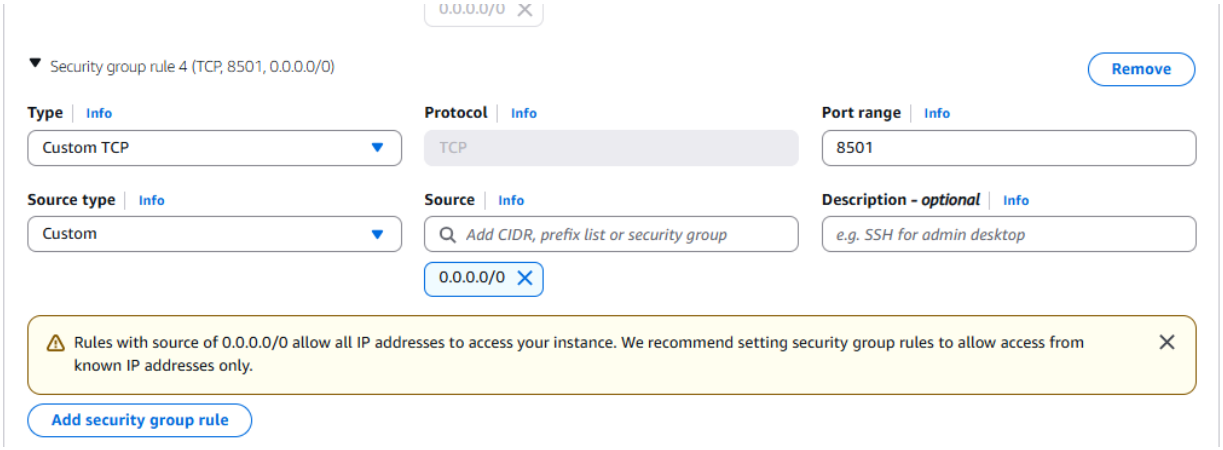

Figure 1 creating custom TCP rule for Streamlit

6. Click Launch Instance.

Connect to the machine and run these commands to install some dependencies:

```
sudo apt update && sudo apt upgrade -y
sudo apt install python3-pip -y
```

Create virtual environment

```
sudo apt install python3-venv -y
python3 -m venv venv
source venv/bin/activate
```

install some required packages for the app

```
pip3 install streamlit pandas matplotlib plotly boto3 numpy joblib capstone
```

make this folder structure

```
Fleet_Management
    app.py
    app_pages
        bootloader_overview.py
        fota.py
        introduction.py
        lsm_overview.py
        network_overview.py
        obd_overview.py
        someip_overview.py
    Bootloader
        cloud.py
        features.csv
        label_encoder.pkl
         malware_detector_model.pkl
    JSONS
        Firmware_v1.json
        Firmwasre_v2.json
        Firmware_v3.json
Flask_Server
    received.py
    JSONS
        File1.json
        File2.json
        File3.json
```

Introduction.py

Purpose: This contains introduction for the pages in the Web app [3]

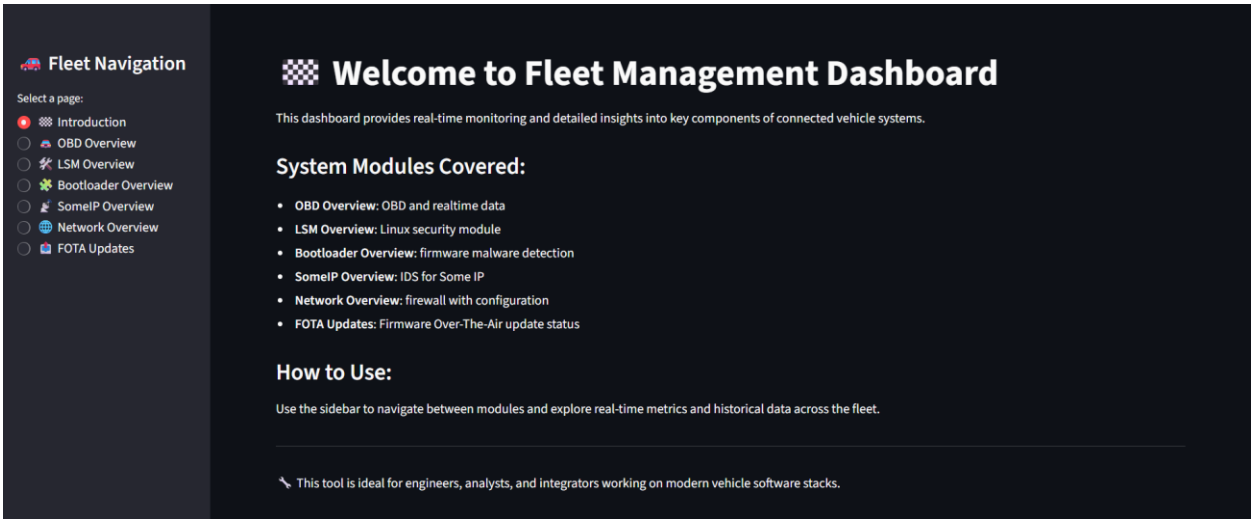

Figure 2 Introduction page

fota.py

Purpose: Allows users to upload firmware updates (in .bin format). [3]

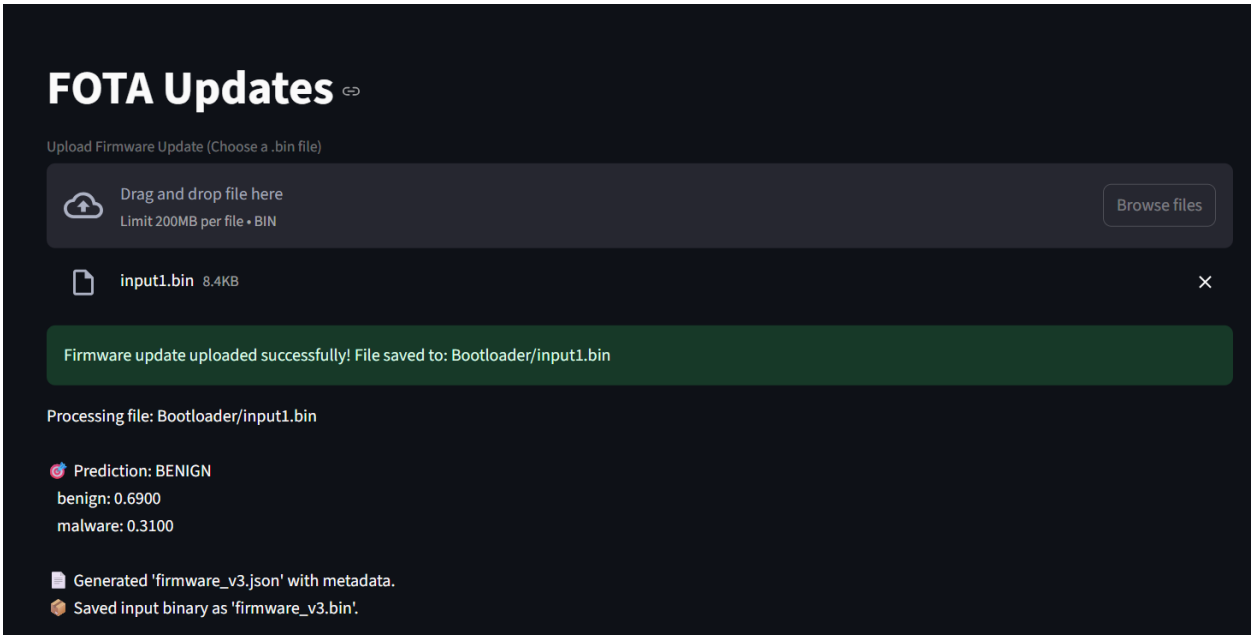

Figure 3 FOTA Updates page

Now to get the .bin file on our machine, run this command:

```
scp -i C:\Users\future\Desktop\WebApp\Web_app.pem
ubuntu@13.58.250.233:~/uploads/debug_sample.bin .
```

This command will securely transfer the file debug_sample.bin from the remote server located at 3.139.97.26(cloud server). The transfer will use the private key specified (Web_app.pem) for secure SSH authentication.

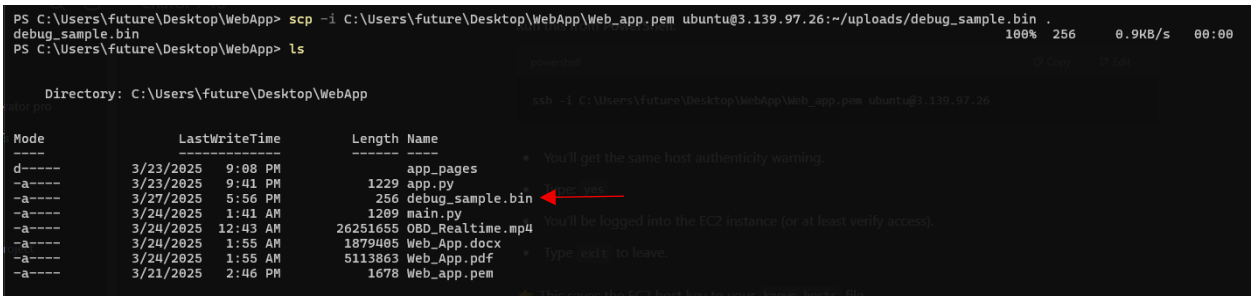

Figure 4 downloaded bin file

app.py

Purpose: This is the main app that serves as the central point for navigation within the Fleet Management Web App.

Used Libraries: Streamlit for the web interface, includes the various page modules like obd_overview, lsm_overview, bootloader_overview, etc. [3]

Run app.py

```
streamlit run app.py --server.port 8501
```

Open the app using this URL

```
http://<EC2_public_ip>:8501/

http://13.58.250.233:8501/
```

Now we need the app to Keep Running on the cloud machine, so we will use nohup command:

```
nohup streamlit run app.py --server.port 8501 &
nohup python3 receiver.py &
```

We can stop the Streamlit app by killing the process using the port it's running on (8501):

1- The lsof (List Open Files) command helps identify the PID of the app running on a specific port (8501).

```
lsof -i :8501
```

2- Then kill it with its PID resulted from the previous command

```
kill <PID>
```

Sending Real-Time data to the Web App

1. Create a Timestream Database

Go to AWS Console → **Timestream**.

Click **"Create database"**.

Name it: FleetManagementDB

Choose: **Standard database** → Click **Create**

2. Create a Timestream Table

In FleetManagementDB, click "Create table"

Name it: OBDTable

Leave the default options as it is

Click Create table

3. Create a Thing (OBD Device)

Thing name: demo_thing

Attach this Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "iot:Publish",
        "iot:Subscribe",
        "iot:Connect",
        "iot:Receive"
      ],
      "Resource": "*"
    }
  ]
}
```

4. Create IoT Rule to Store data in Timestream

Name the Role: OBDThingtoTimeStream

Enter this SQL statement to get all the messages from the OBD topic.

```
SELECT * FROM 'obd_topic'
```

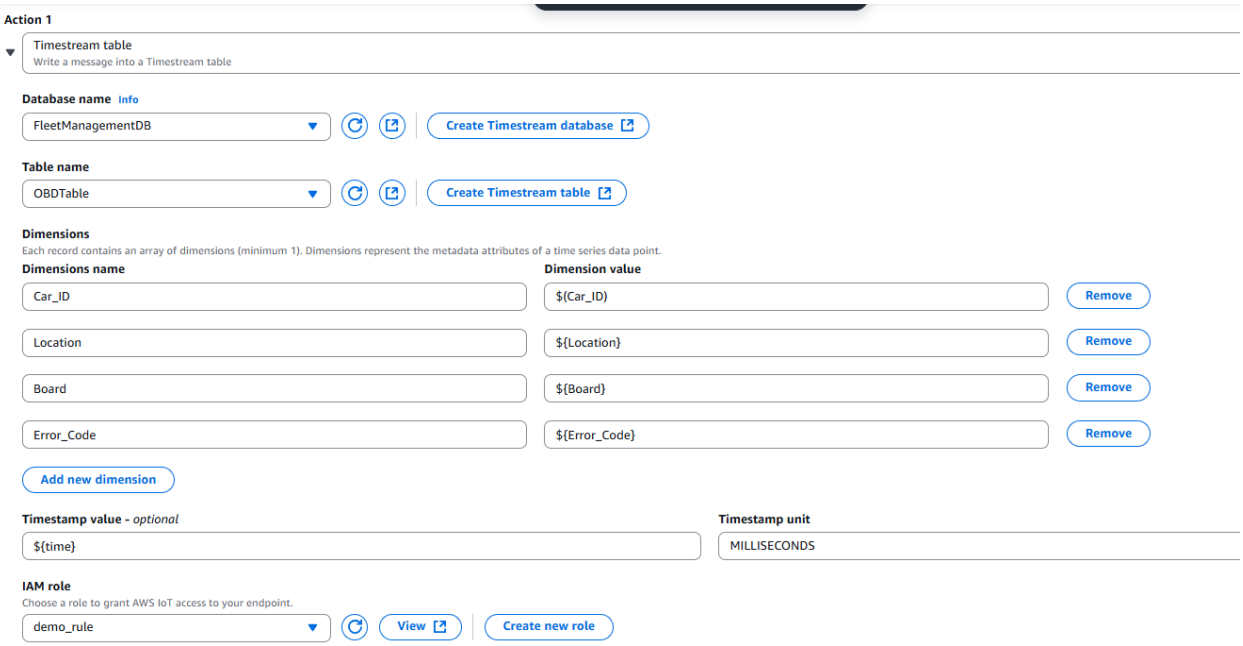

Figure 5 Role parameters

5. Attach IAM Role to EC2 (Timestream Read Access)

Create an IAM Role (EC2TimestreamReader)

Attach this permission:

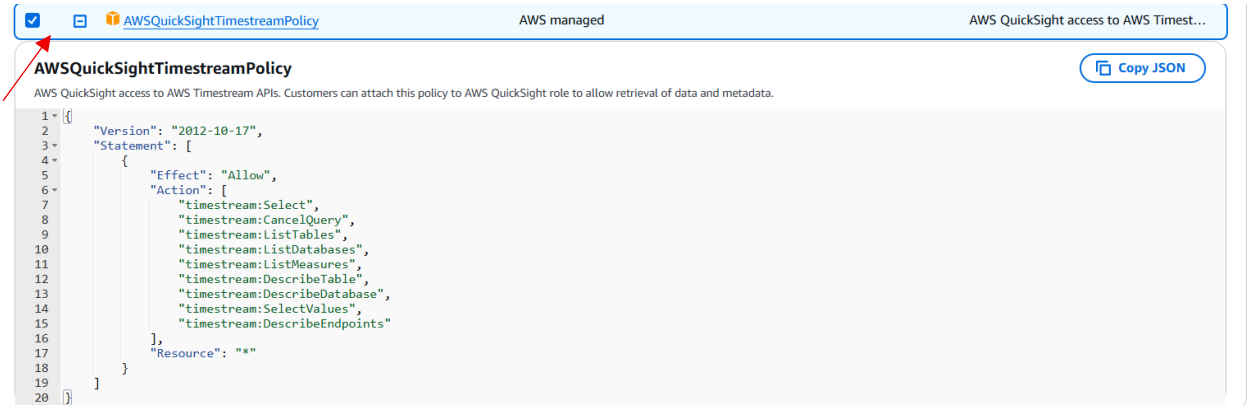

Figure 6 Policy for EC2 instance

Then to Attach this role to the EC2 instance:

1- under **Instance Settings**

2- Click Actions → Security → Modify IAM Role.

3- Select the IAM role that we have just created (EC2TimestreamReader).

4- Click Update IAM role.

Test the Fleet Management App

1- Run the C++ app that publish messages to AWS IOTCore Service

2- Apply MQTT test and make sure that messages are published correctly

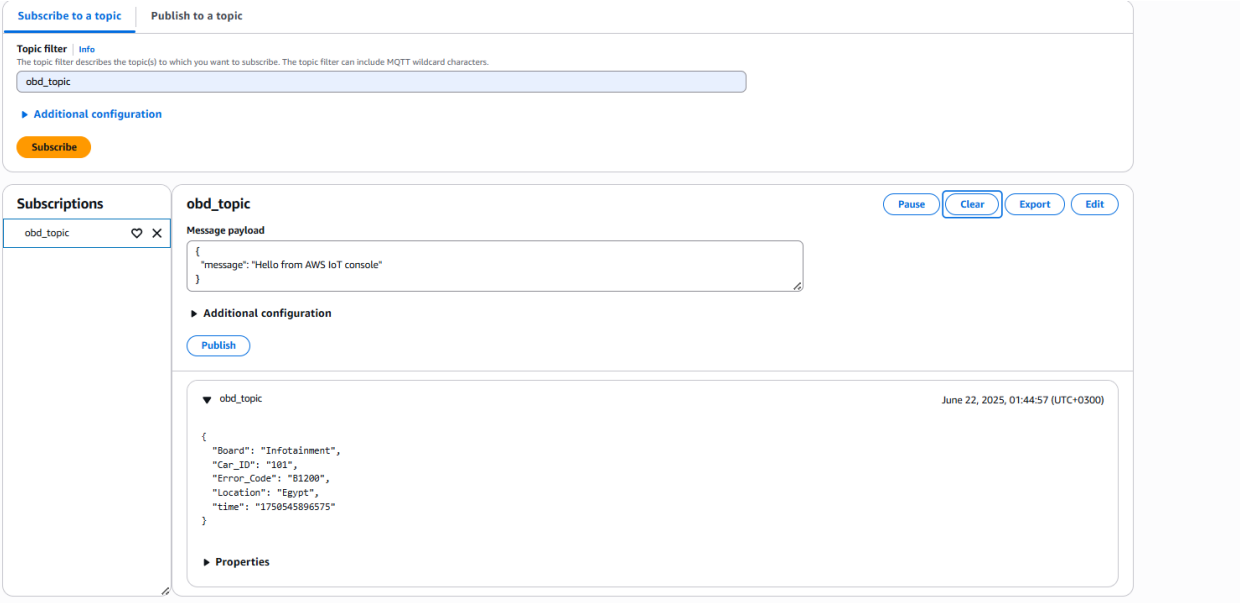

Figure 7 MQTT test client applied on OBD topic

Apply this Querry test to make sure that Created role (transfer messages from IOTCore to Amazon Timestream DB) works properly:

```
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'Location' THEN measure_value::varchar END) AS
Location,
    MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board,
    MAX(CASE WHEN measure_name = 'Error_Code' THEN measure_value::varchar END) AS
Error_Code
FROM "FleetManagementDB"."OBDTable"
GROUP BY time
ORDER BY time DESC
```

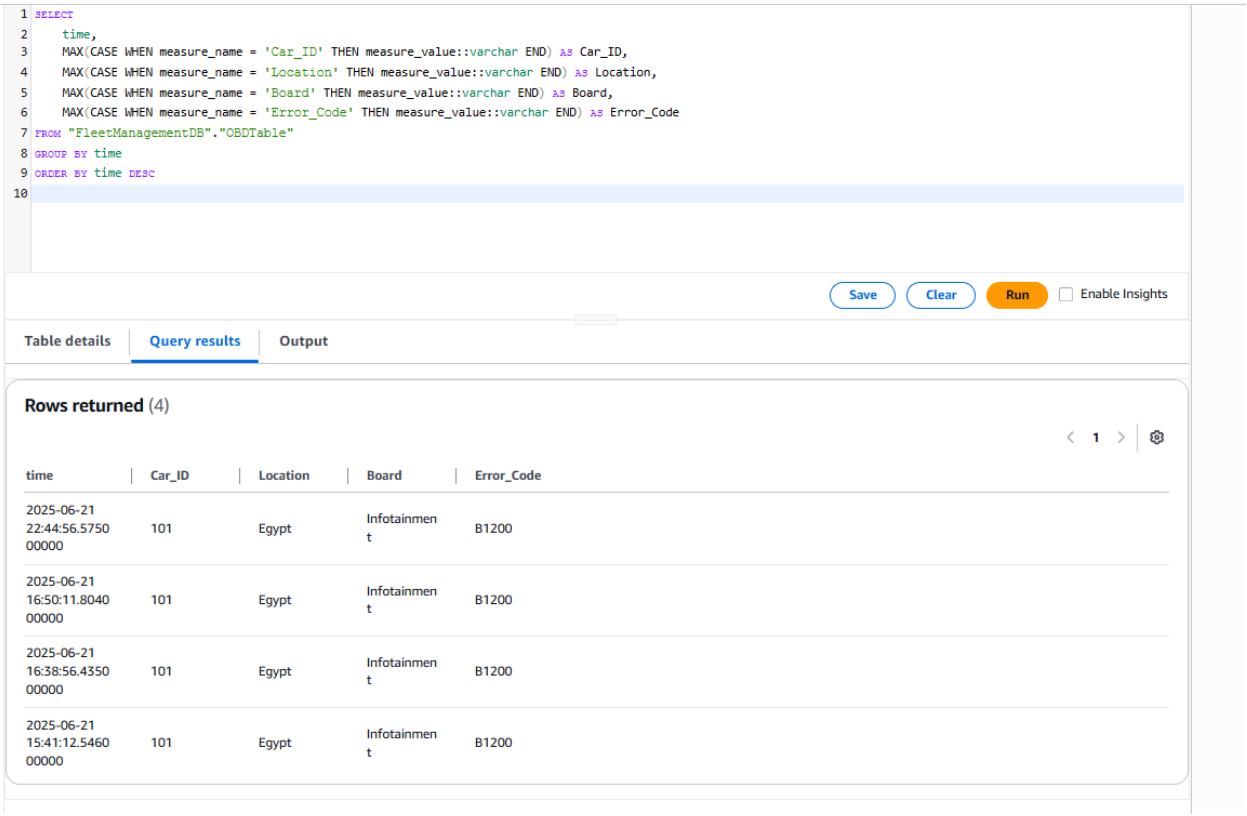

Figure 8 Querry Results

Then we have created a script that creates a dashboard that displays live OBD error data retrieved from AWS Timestream database. It fetches the latest OBD logs, including car ID, location, error code, and timestamp, board type, and displays them in a tabular format with expandable entries for each log. [3]

Used Libraries:

Streamlit: For creating the web app interface and displaying the data.

boto3: For interacting with AWS Timestream, querying the database, and fetching the data.

pandas: for data handling.

```
import streamlit as st
import boto3
import pandas as pd
from botocore.exceptions import BotoCoreError, ClientError

st.title("🚗 Live OBD Error Dashboard")

# Initialize Timestream client
try:
```

```
    client = boto3.client('timestream-query', region_name='us-east-2')
except (BotoCoreError, ClientError) as e:
    st.error(f"Failed to create Timestream client: {e}")
    st.stop()

# SQL Query
QUERY = """
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'Location' THEN measure_value::varchar END) AS
Location,
    MAX(CASE WHEN measure_name = 'Error_Code' THEN measure_value::varchar END) AS
Error_Code
FROM "FleetManagementDB"."FleetManagementDBTable"
GROUP BY time
ORDER BY time DESC
LIMIT 50
"""

# Execute query
try:
    response = client.query(QueryString=QUERY)
    rows = response.get('Rows', [])
    columns = [col['Name'] for col in response.get('ColumnInfo', [])]

    if not rows or not columns:
        st.warning("No data returned from Timestream.")
        st.stop()

    # Parse rows
    data = [[col.get('ScalarValue', '') for col in row['Data']] for row in rows]
    df = pd.DataFrame(data, columns=columns)

except (BotoCoreError, ClientError) as e:
    st.error(f"Query failed: {e}")
    st.stop()

# Display table
st.subheader("🔍 Latest OBD Logs")
st.dataframe(df)

# Expanders for details
for _, row in df.iterrows():
    with st.expander(f"🚘 {row['Car_ID']} - {row['Error_Code']}"):
        st.write(f"📍 **Location:** {row['Location']}")
```

```
    st.write(f"🗓️ **Time:** {row['time']}")
```

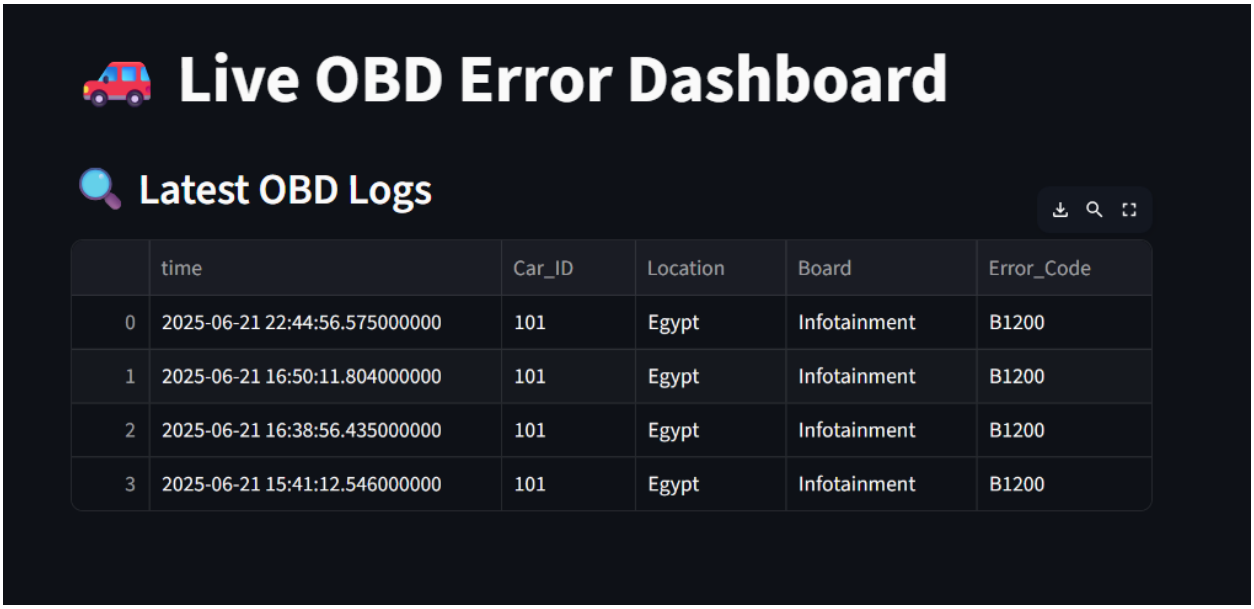

Figure 9 Final Results

The following link provides a demonstration video showcasing the application in operation:

https://drive.google.com/file/d/1qDM56tukEz2kaKOdhhmk4qCfGnKTPpnH/view?usp=sharing

Creating multiple things for system components

Sending LSM Real-Time data to the Web App

1. Choose FleetManagementDB

2. Create a Timestream Table

In FleetManagementDB, click "Create table"

Name it: LSMTable

Leave the default options as it is

Click Create table

3. Create a Thing (LSM app)

Thing name: LSM_thing

Attach Iot Policy

4. Create IoT Rule to Store data in Timestream

Name the Role: LSMThingtoTimeStream

Enter this SQL statement to get all the messages from the LSM topic.

```
SELECT * FROM 'lsm_topic'
```

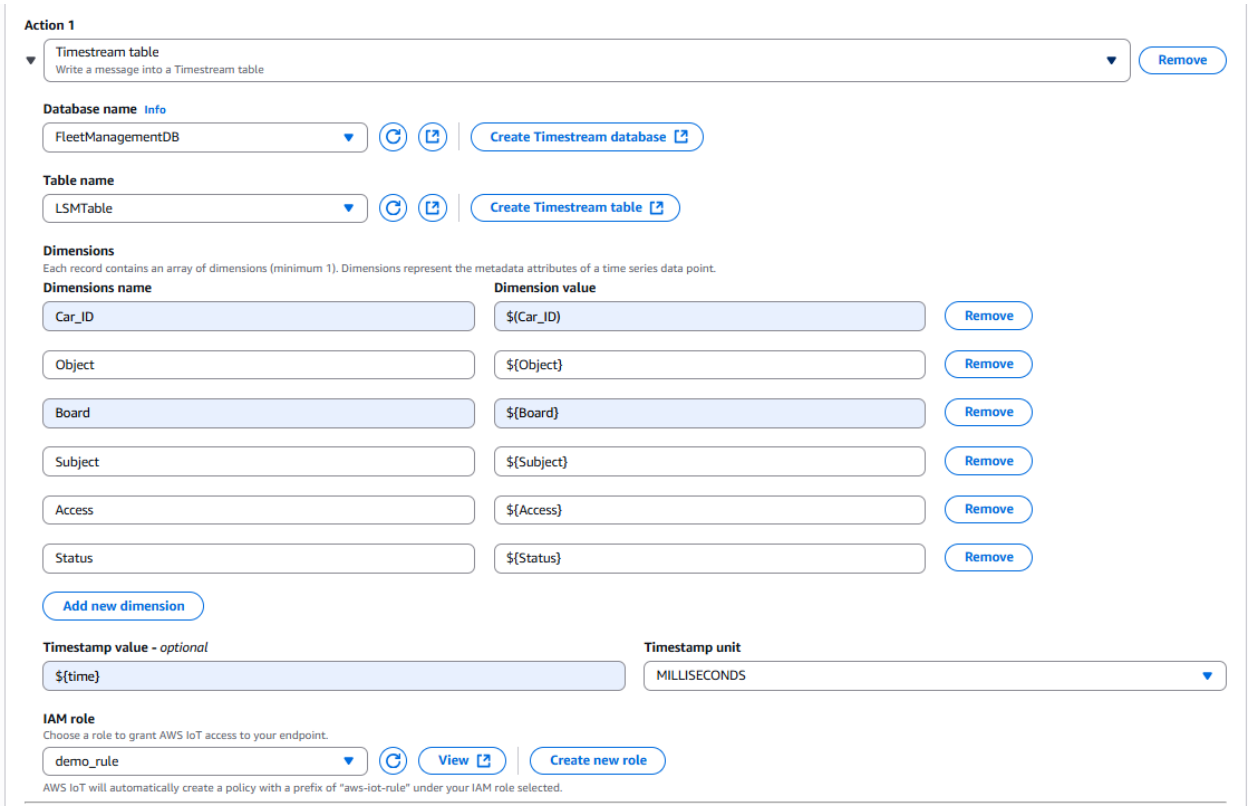

Figure 10 Role parameters

Test the Fleet Management App

3- Run the C++ app that publish messages to AWS IOTCore Service

4- Apply MQTT test and make sure that messages are published correctly

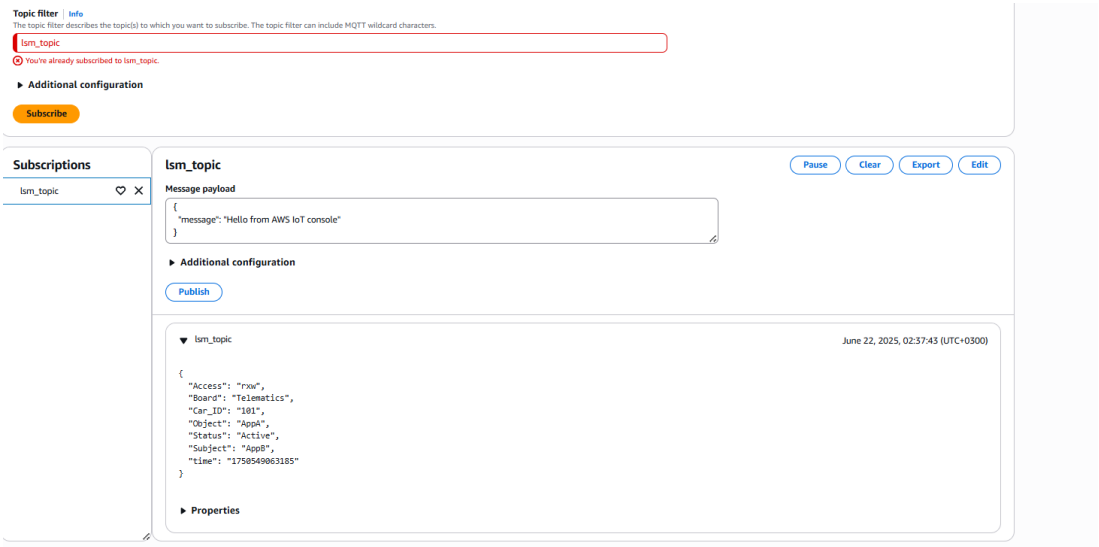

Figure 11 MQTT test client

Apply this Querry test to make sure that Created role (transfer messages from IOTCore to Amazon Timestream DB) works properly:

```
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'Object' THEN measure_value::varchar END) AS Object,
    MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board,
    MAX(CASE WHEN measure_name = 'Subject' THEN measure_value::varchar END) AS Subject,
    MAX(CASE WHEN measure_name = 'Access' THEN measure_value::varchar END) AS Access,
    MAX(CASE WHEN measure_name = 'Status' THEN measure_value::varchar END) AS Status
FROM "FleetManagementDB"."LSMTable"
GROUP BY time
ORDER BY time DESCGROUP BY time
ORDER BY time DESC
```

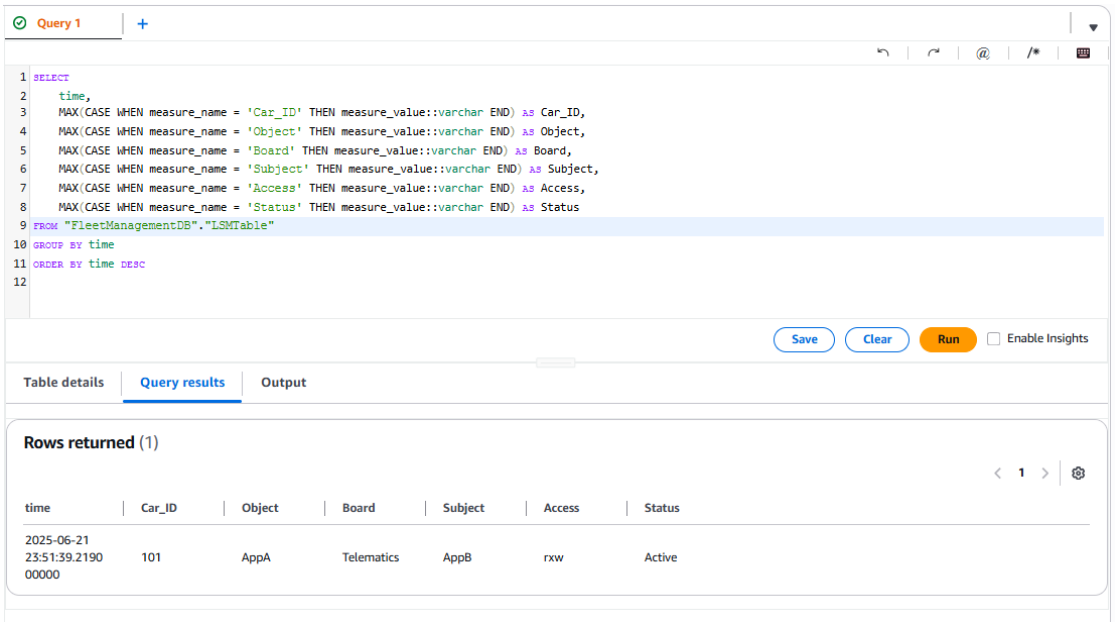

Figure 12 Querry Results

Then we have created a script that creates a dashboard that displays live LSM data retrieved from AWS Timestream database.

```
import streamlit as st
import boto3
import pandas as pd
from botocore.exceptions import BotoCoreError, ClientError

st.title("🚗 Live LSM Dashboard")

# Initialize Timestream client
try:
    client = boto3.client('timestream-query', region_name='us-east-2')
except (BotoCoreError, ClientError) as e:
    st.error(f"Failed to create Timestream client: {e}")
    st.stop()

# SQL Query
QUERY = """
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'Object' THEN measure_value::varchar END) AS Object,
    MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board,
    MAX(CASE WHEN measure_name = 'Subject' THEN measure_value::varchar END) AS Subject,
    MAX(CASE WHEN measure_name = 'Access' THEN measure_value::varchar END) AS Access,
    MAX(CASE WHEN measure_name = 'Status' THEN measure_value::varchar END) AS Status
FROM "FleetManagementDB"."LSMTable"
```

```
GROUP BY time
ORDER BY time DESC
"""

# Execute query
try:
    response = client.query(QueryString=QUERY)
    rows = response.get('Rows', [])
    columns = [col['Name'] for col in response.get('ColumnInfo', [])]

    if not rows or not columns:
        st.warning("No data returned from Timestream.")
        st.stop()

    # Parse rows
    data = [[col.get('ScalarValue', '') for col in row['Data']] for row in rows]
    df = pd.DataFrame(data, columns=columns)

except (BotoCoreError, ClientError) as e:
    st.error(f"Query failed: {e}")
    st.stop()

# Display table
st.subheader("🔍 Latest LSM Logs")
st.dataframe(df)
```

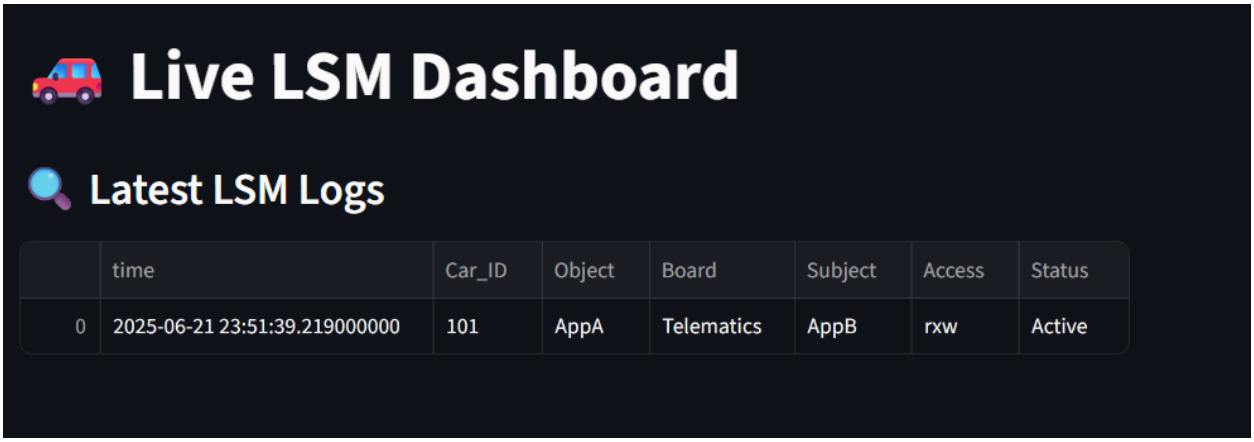

Figure 13 Final Results

Sending BootLoader Real-Time data to the Web App

1. Choose FleetManagementDB

2. Create a Timestream Table

In FleetManagementDB, click "Create table"

Name it: BLTable

Leave the default options as it is

Click Create table

3. Create a Thing (BL app)

Thing name: BL_thing

Attach Iot Policy

4. Create IoT Rule to Store data in Timestream

Name the Role: BLThingtoTimeStream

Enter this SQL statement to get all the messages from the BL topic.

```
SELECT * FROM 'BL_topic'
```

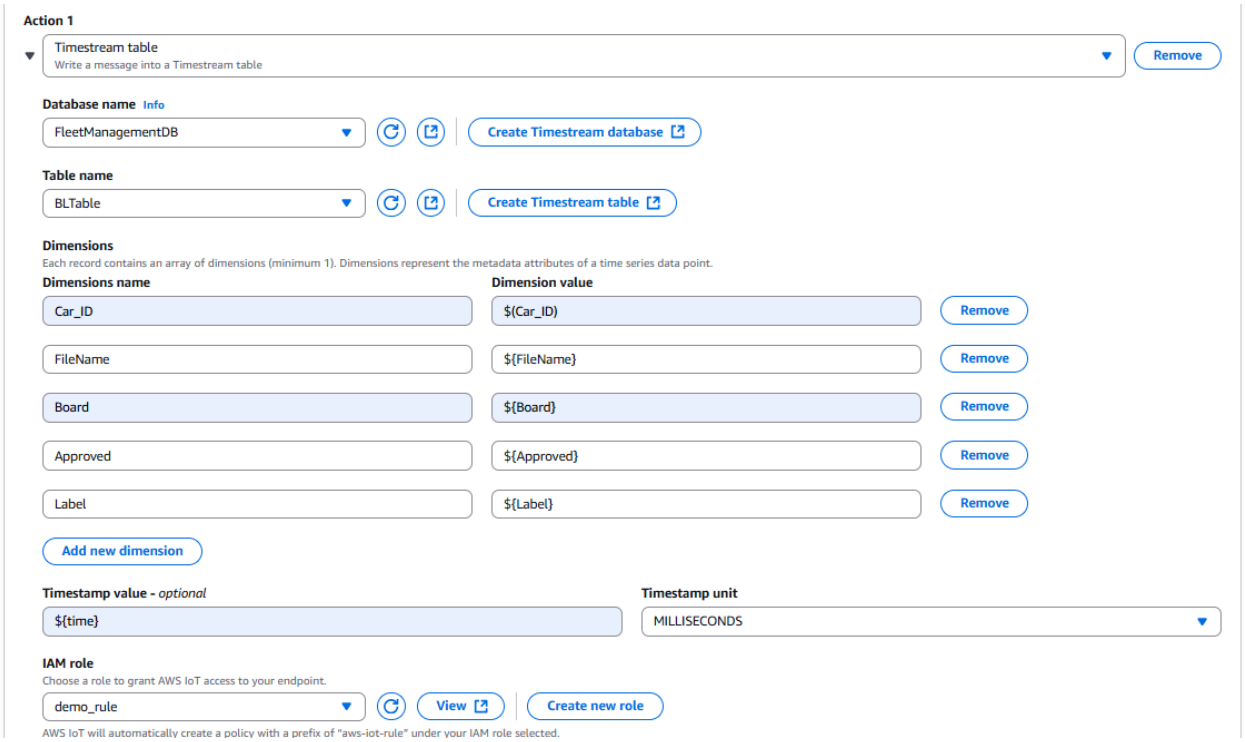

Figure 14 Role parameters

Test the Fleet Management App

1- Run the C++ app that publish messages to AWS IOTCore Service

2- Apply MQTT test and make sure that messages are published correctly

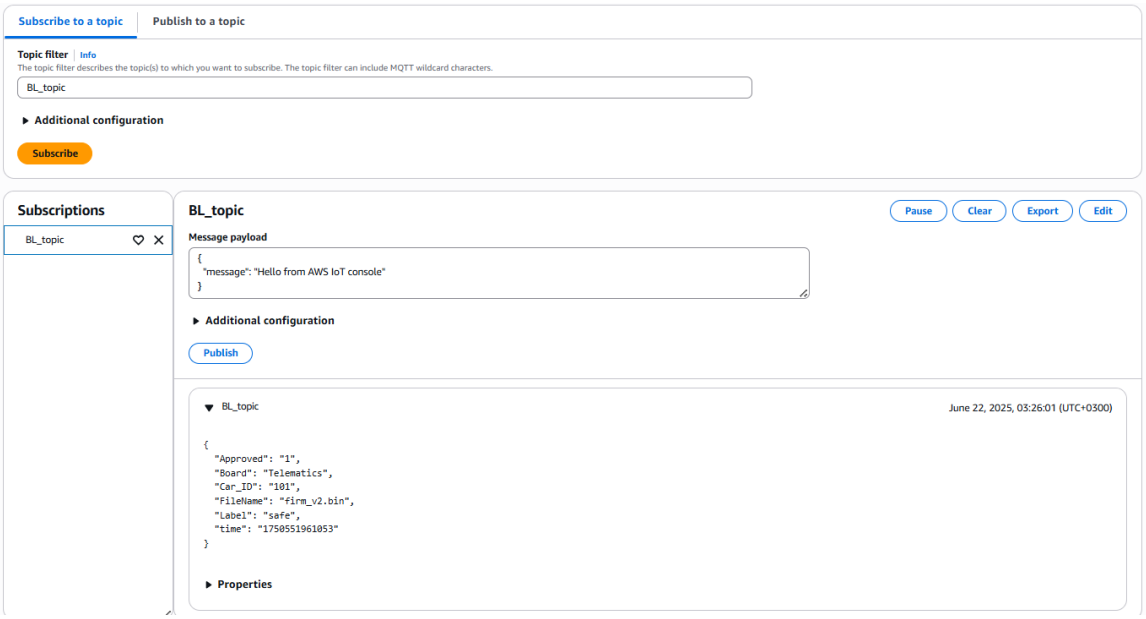

Figure 15 MQTT test client

Apply this Querry test to make sure that Created role (transfer messages from IOTCore to Amazon Timestream DB) works properly:

```
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'FileName' THEN measure_value::varchar END) AS
FileName,
    MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board,
    MAX(CASE WHEN measure_name = 'Approved' THEN measure_value::varchar END) AS
Approved,
    MAX(CASE WHEN measure_name = 'Label' THEN measure_value::varchar END) AS Label
FROM "FleetManagementDB"."BLTable"
GROUP BY time
ORDER BY time DESC
```

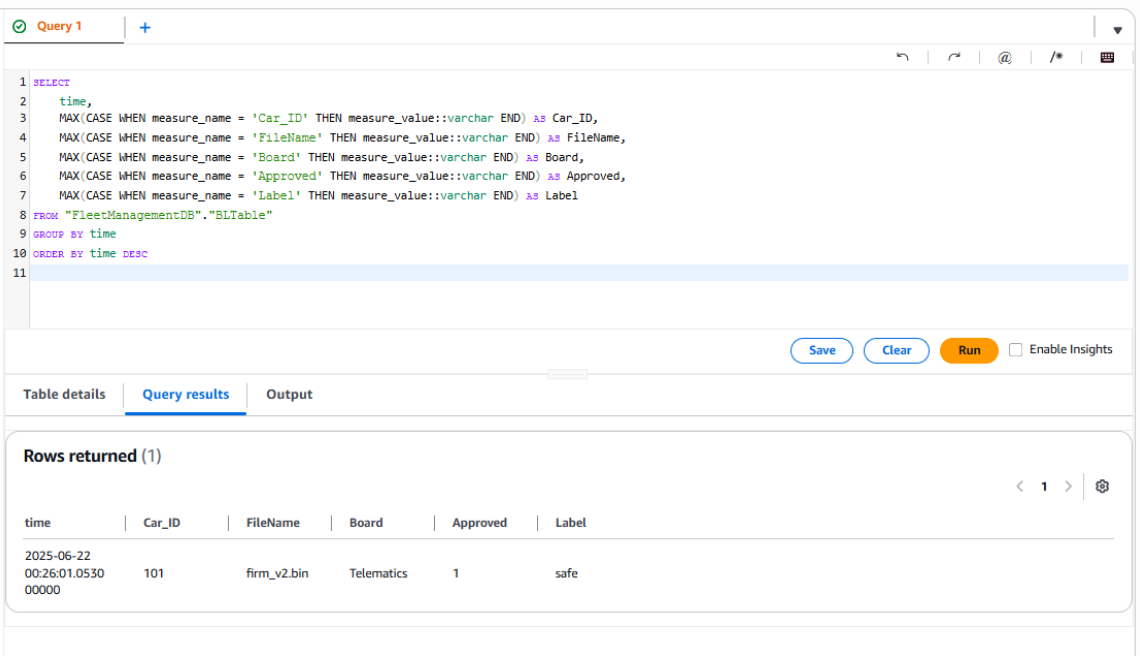

Figure 16 Querry Results

Then we have created a script that creates a dashboard that displays live Bootloader data retrieved from AWS Timestream database.

```
import streamlit as st
import boto3
import pandas as pd
from botocore.exceptions import BotoCoreError, ClientError

st.title("🚗 Live BootLoader Dashboard")

# Initialize Timestream client
try:
    client = boto3.client('timestream-query', region_name='us-east-2')
except (BotoCoreError, ClientError) as e:
    st.error(f"Failed to create Timestream client: {e}")
    st.stop()

# SQL Query
QUERY = """
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'FileName' THEN measure_value::varchar END) AS
FileName,
    MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board,
    MAX(CASE WHEN measure_name = 'Approved' THEN measure_value::varchar END) AS
Approved,
```

```
    MAX(CASE WHEN measure_name = 'Label' THEN measure_value::varchar END) AS Label
FROM "FleetManagementDB"."BLTable"
GROUP BY time
ORDER BY time DESC
"""

# Execute query
try:
    response = client.query(QueryString=QUERY)
    rows = response.get('Rows', [])
    columns = [col['Name'] for col in response.get('ColumnInfo', [])]

    if not rows or not columns:
        st.warning("No data returned from Timestream.")
        st.stop()

    # Parse rows
    data = [[col.get('ScalarValue', '') for col in row['Data']] for row in rows]
    df = pd.DataFrame(data, columns=columns)

except (BotoCoreError, ClientError) as e:
    st.error(f"Query failed: {e}")
    st.stop()

# Display table
st.subheader("🔍 Latest BootLoader Logs")
st.dataframe(df)
```

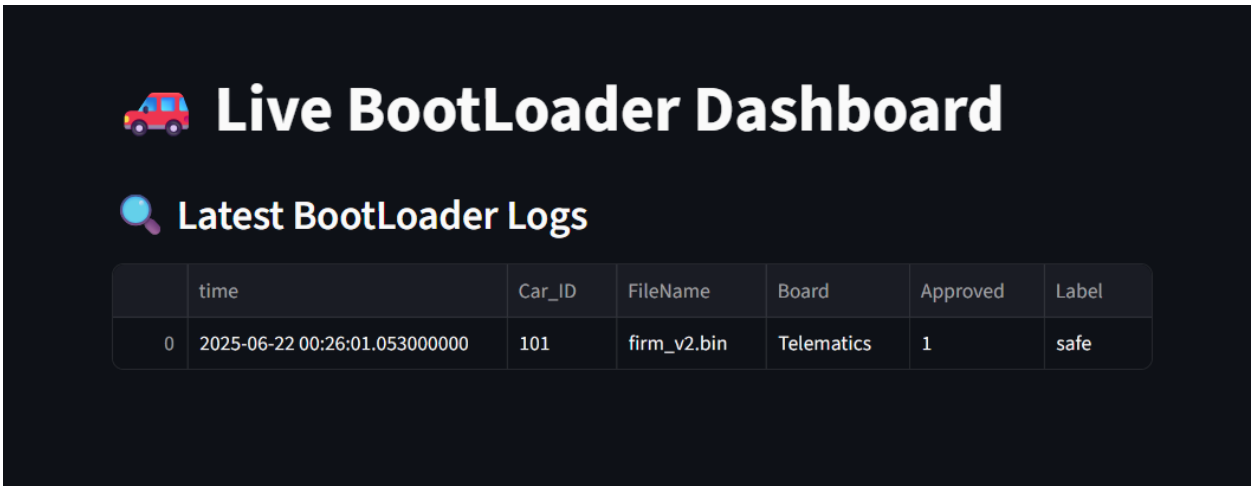

Figure 17 Final Results

Sending SomeIP 1 Real-Time data to the Web App

1. Choose FleetManagementDB

2. Create a Timestream Table

In FleetManagementDB, click "Create table"

Name it: SIPOneTable

Leave the default options as it is

Click Create table

3. Create a Thing (SIPOne app)

Thing name: SIPOne_thing

Attach Iot Policy

4. Create IoT Rule to Store data in Timestream

Name the Role: SIPOneThingtoTimeStream

Enter this SQL statement to get all the messages from the SomeIPone topic.

```
SELECT * FROM 'SIPOne_topic'
```

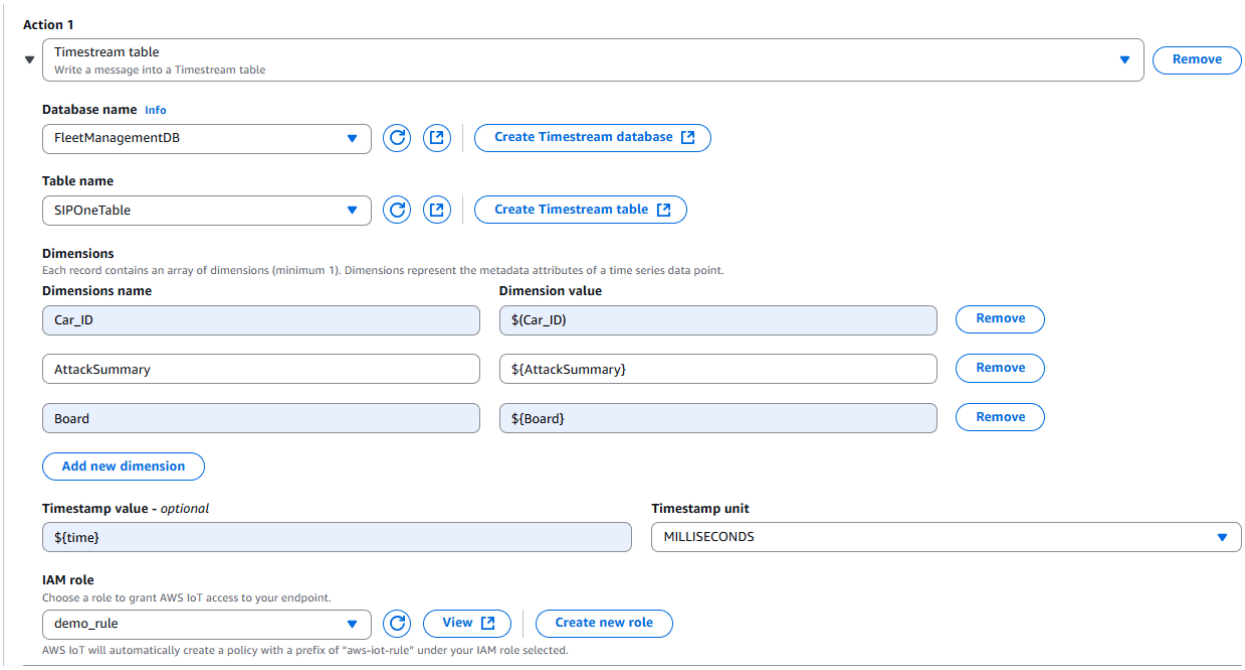

Figure 18 Role parameters

Test the Fleet Management App

1- Run the C++ app that publish messages to AWS IOTCore Service

2- Apply MQTT test and make sure that messages are published correctly

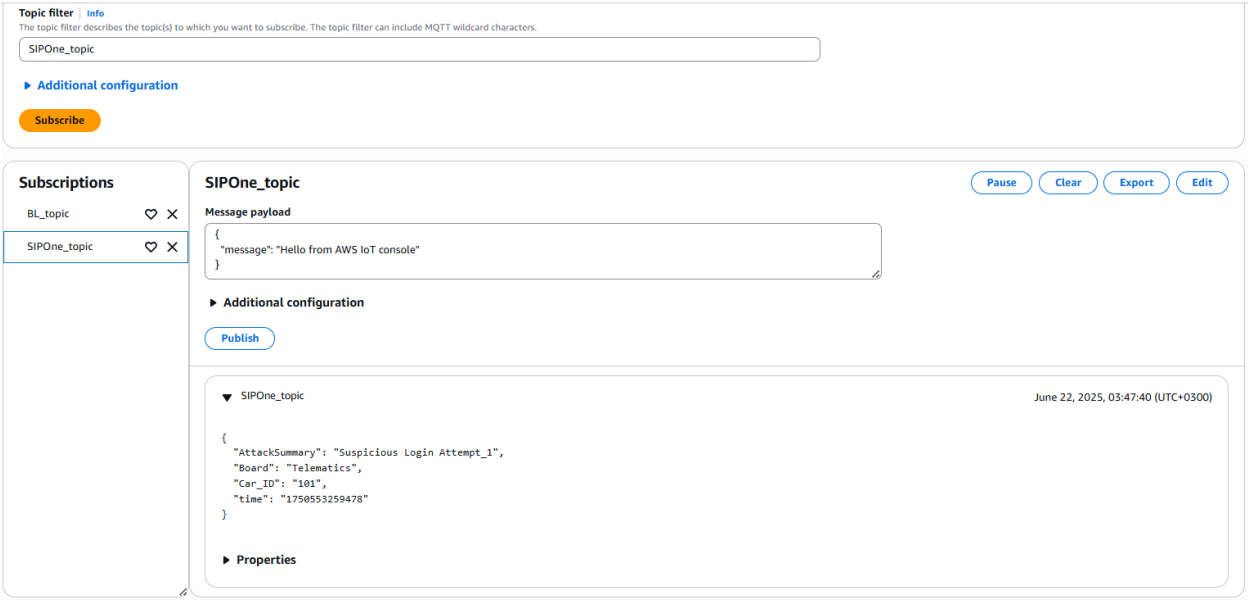

Figure 19 MQTT test client

Apply this Querry test to make sure that Created role (transfer messages from IOTCore to Amazon Timestream DB) works properly:

```
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END) AS
AttackSummary,
    MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
FROM "FleetManagementDB"."SIPOneTable"
GROUP BY time
ORDER BY time DESC
```

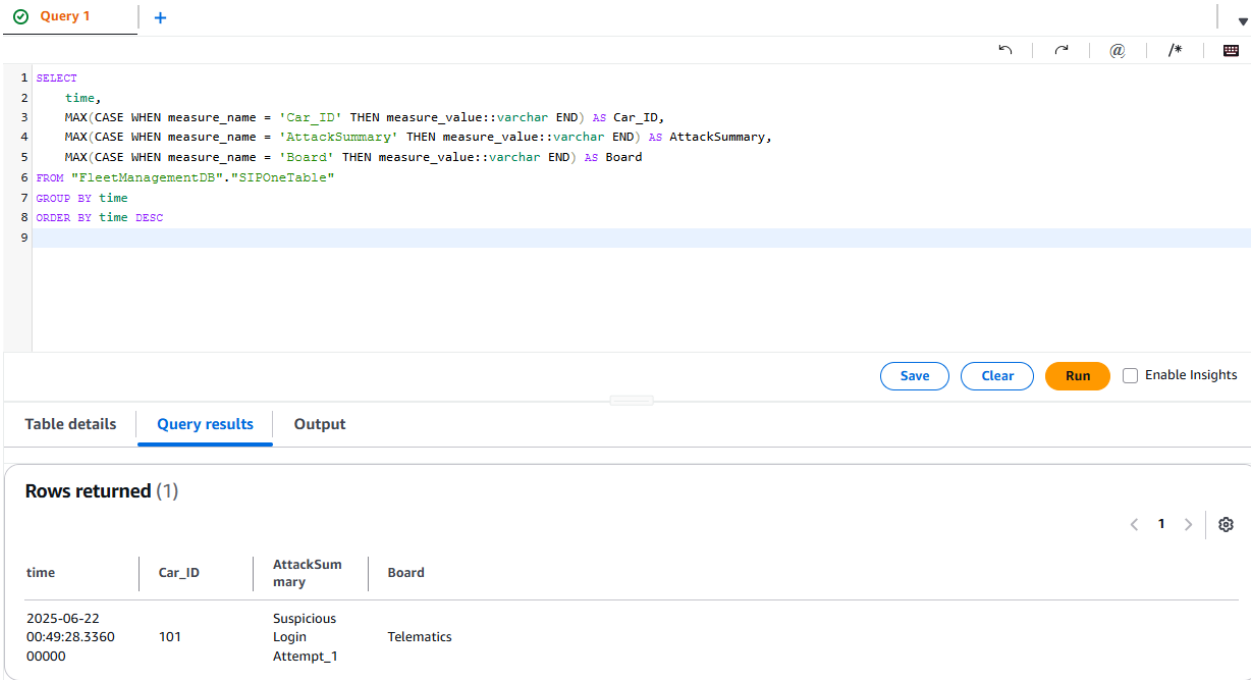

Figure 20 Querry Results

Sending SomeIP 2 Real-Time data to the Web App

1. Choose FleetManagementDB

2. Create a Timestream Table

In FleetManagementDB, click "Create table"

Name it: SIPTwoTable

Leave the default options as it is

Click Create table

3. Create a Thing (SIPTwo Device)

Thing name: SIPTwo_thing

Attach Iot Policy

4. Create IoT Rule to Store data in Timestream

Name the Role: SIPTwoThingtoTimeStream

Enter this SQL statement to get all the messages from the SomeIP 2 topic.

```
SELECT * FROM 'SIPTwo_topic'
```

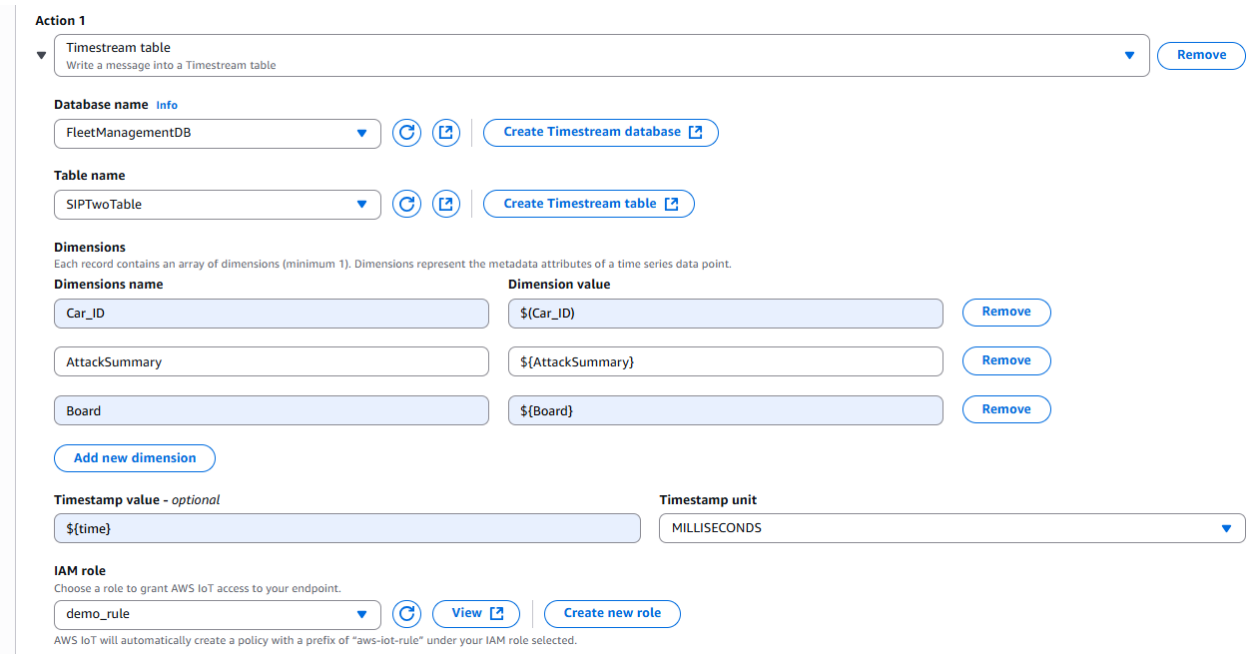

Figure 21 Role parameters

Test the Fleet Management App

1- Run the C++ app that publish messages to AWS IOTCore Service

2- Apply MQTT test and make sure that messages are published correctly

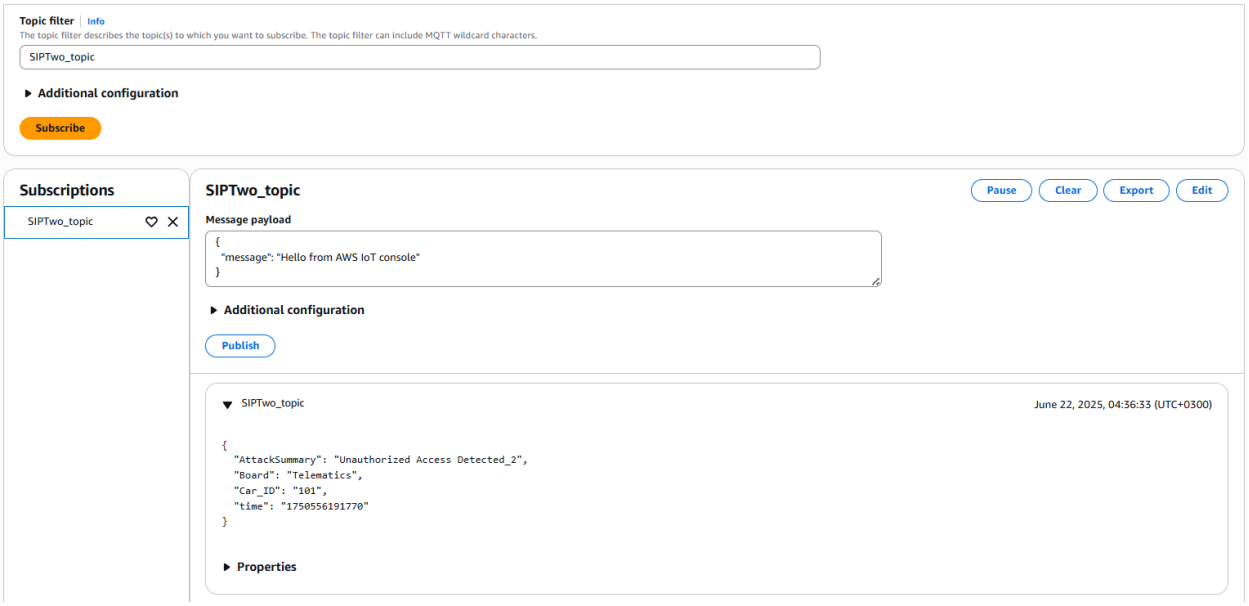

Figure 22 MQTT test client

Apply this Querry test to make sure that Created role (transfer messages from IOTCore to Amazon Timestream DB) works properly:

```
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END) AS
AttackSummary,
    MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
FROM "FleetManagementDB"."SIPOneTable"
GROUP BY time
ORDER BY time DESC
```

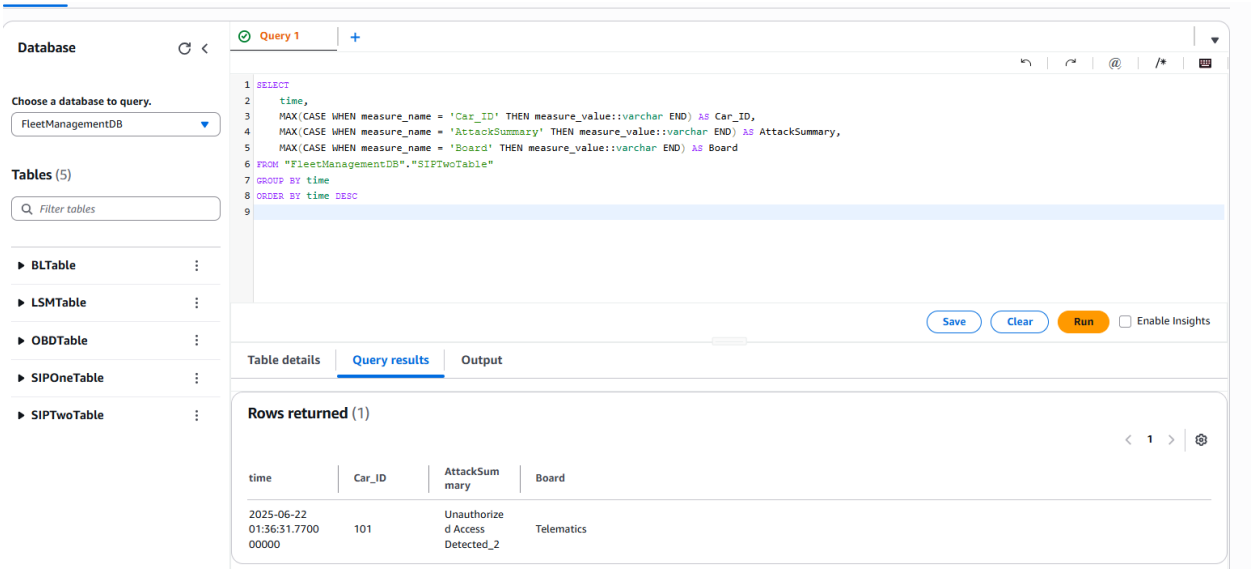

Figure 23 Querry Results

Then we have created a script that creates a dashboard that displays live SomeIP 1 & SomeIP 2 data retrieved from AWS Timestream database.

```
import streamlit as st
import boto3
import pandas as pd
from botocore.exceptions import BotoCoreError, ClientError

st.title("🚗 Live SomeIP Dashboard")

# Initialize Timestream client
try:
    client = boto3.client('timestream-query', region_name='us-east-2')
except (BotoCoreError, ClientError) as e:
```

```
    st.error(f"Failed to create Timestream client: {e}")
    st.stop()

# Define queries
queries = {
    "SOMEIP 1": """
        SELECT
            time,
            MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS
Car_ID,
            MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END)
AS AttackSummary,
            MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
        FROM "FleetManagementDB"."SIPOneTable"
        GROUP BY time
        ORDER BY time DESC
    """,
    "SOMEIP 2": """
        SELECT
            time,
            MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS
Car_ID,
            MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END)
AS AttackSummary,
            MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
        FROM "FleetManagementDB"."SIPTwoTable"
        GROUP BY time
        ORDER BY time DESC
    """
}

# Function to run a Timestream query and return DataFrame
def run_query(query: str):
    try:
        response = client.query(QueryString=query)
        rows = response.get('Rows', [])
        columns = [col['Name'] for col in response.get('ColumnInfo', [])]

        if not rows or not columns:
            return pd.DataFrame()

        data = [[col.get('ScalarValue', '') for col in row['Data']] for row in rows]
        return pd.DataFrame(data, columns=columns)

    except (BotoCoreError, ClientError) as e:
        st.error(f"Query failed: {e}")
        return pd.DataFrame()
```

```
# Query and show data for both SOMEIP tables
for label, sql in queries.items():
    df = run_query(sql)
    st.subheader(f"🔍 Latest {label} Logs")
    if df.empty:
        st.warning(f"No data returned for {label}.")
    else:
        st.dataframe(df)
```

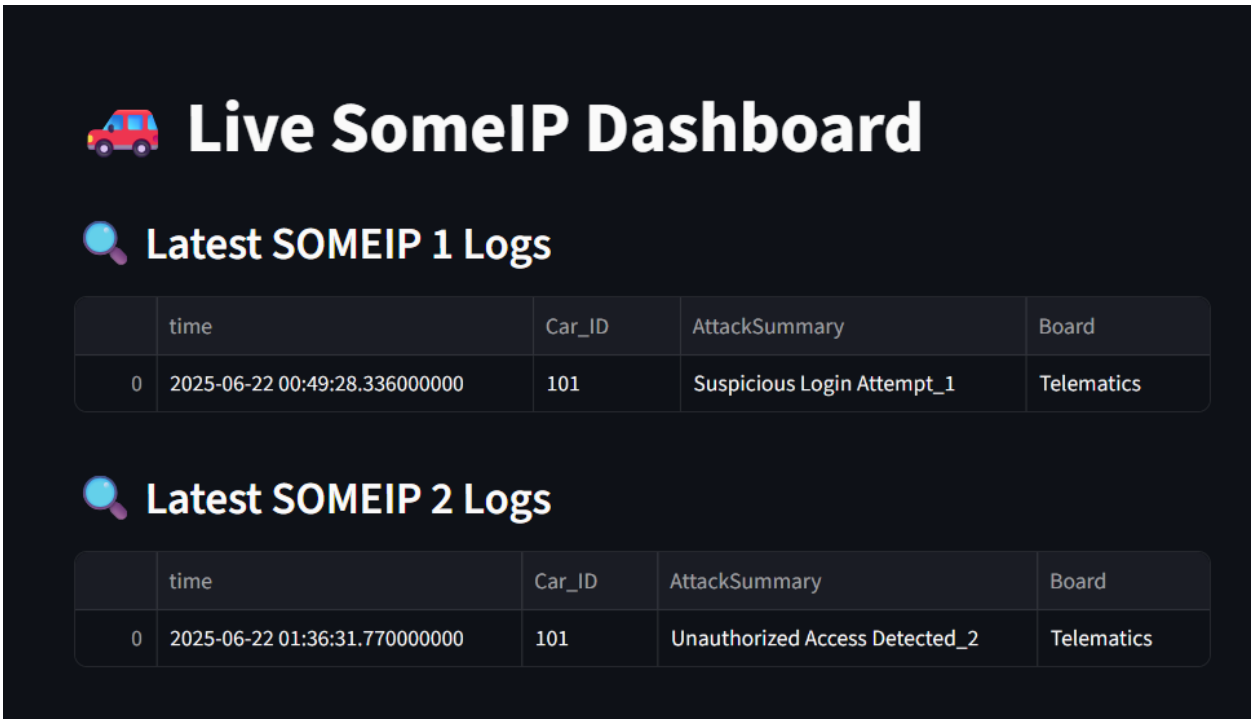

Figure 24 Final Results

Sending Network Real-Time data to the Web App

1. Choose FleetManagementDB

2. Create a Timestream Table

In FleetManagementDB, click "Create table"

Name it: NetworkTable

Leave the default options as it is

Click Create table

3. Create a Thing (Network app)

Thing name: Network_thing

Attach Iot Policy

4. Create IoT Rule to Store data in Timestream

Name the Role: NetworkThingtoTimeStream

Enter this SQL statement to get all the messages from the Network topic.

```
SELECT * FROM 'network_topic'
```

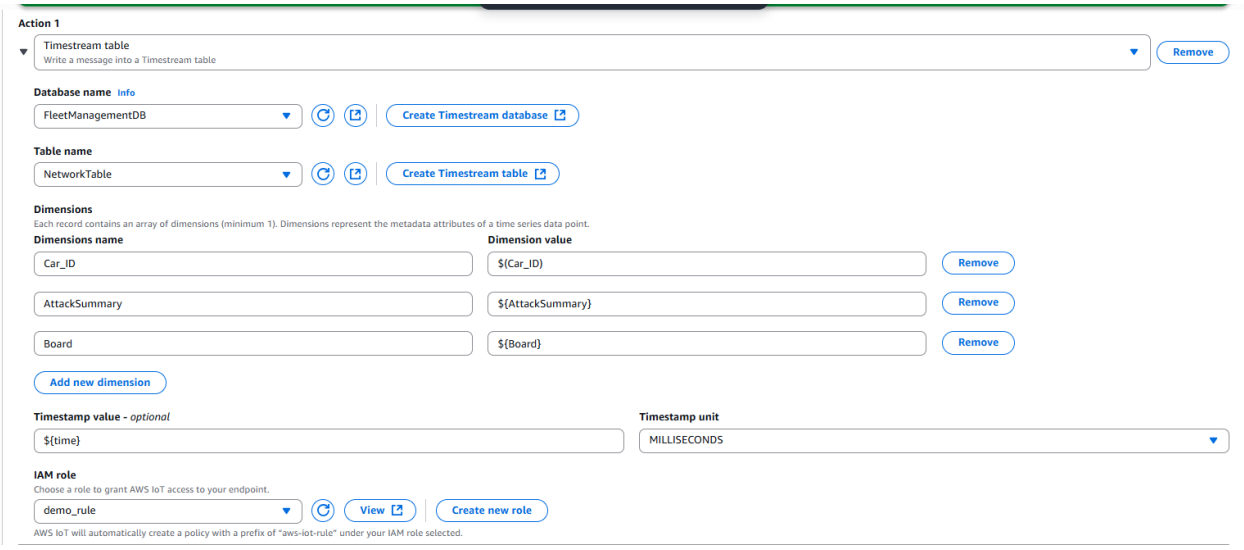

Figure 25 Role parameters

Test the Fleet Management App

1- Run the C++ app that publish messages to AWS IOTCore Service

2- Apply MQTT test and make sure that messages are published correctly

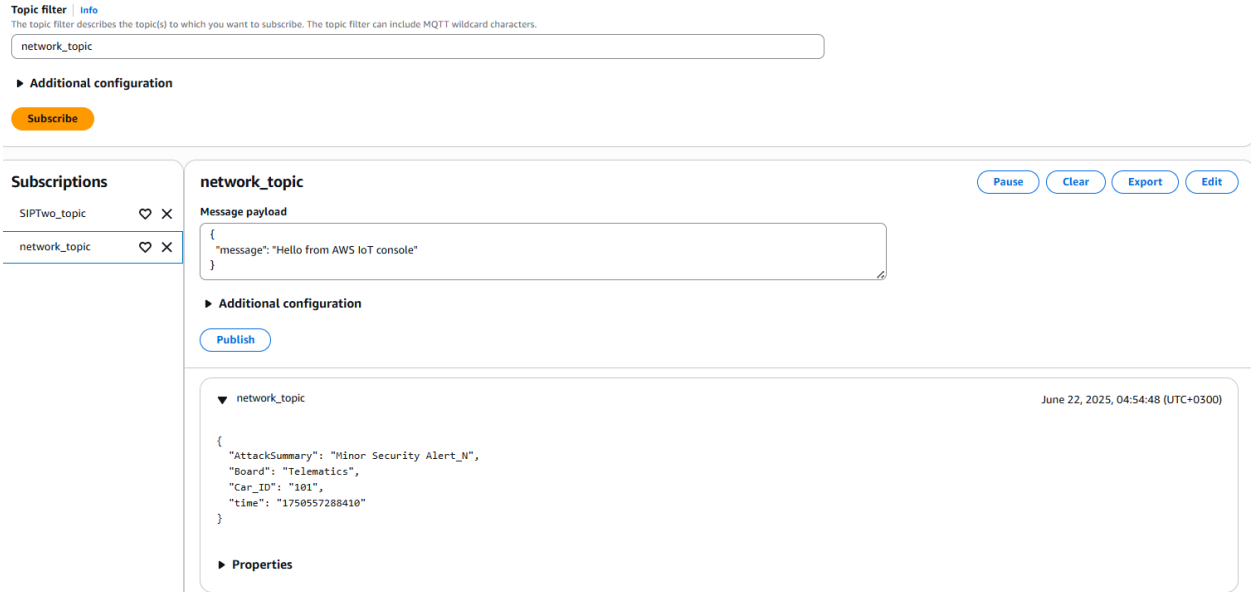

Figure 26 MQTT test client

Apply this Querry test to make sure that Created role (transfer messages from IOTCore to Amazon Timestream DB) works properly:

```
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END) AS
AttackSummary,
    MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
FROM "FleetManagementDB"."NetworkTable"
GROUP BY time
ORDER BY time DESC
```

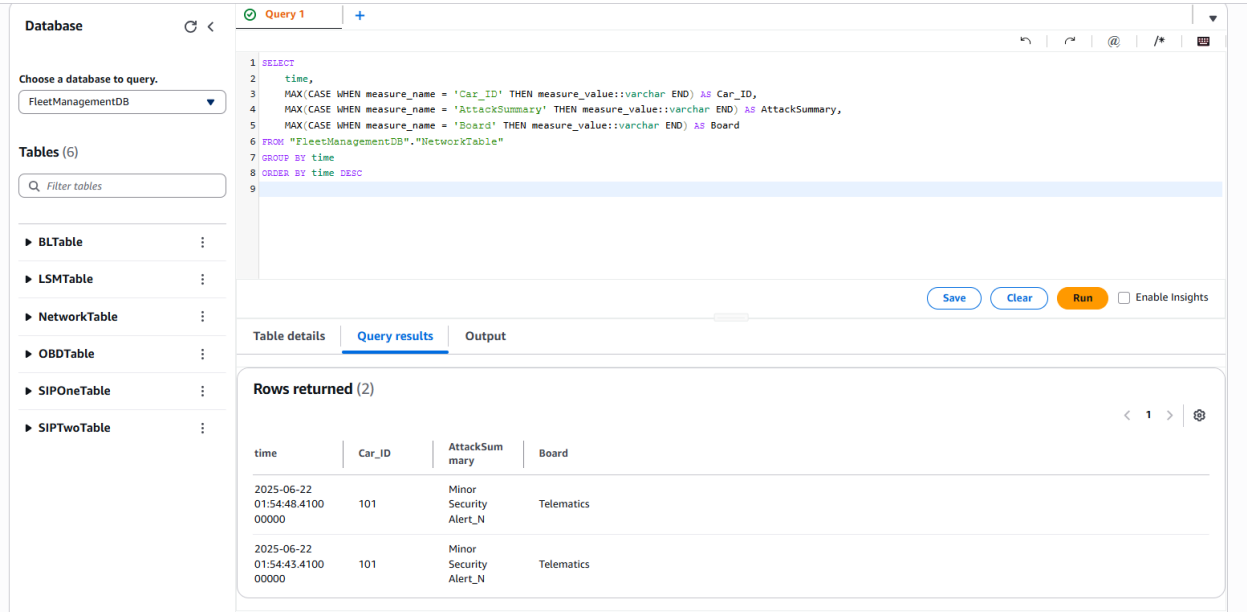

Figure 27 Querry Results

Then we have created a script that creates a dashboard that displays live Netwrok data retrieved from AWS Timestream database.

```
import streamlit as st
import boto3
import pandas as pd
from botocore.exceptions import BotoCoreError, ClientError

st.title("🚗 Live Network Dashboard")

# Initialize Timestream client
try:
    client = boto3.client('timestream-query', region_name='us-east-2')
except (BotoCoreError, ClientError) as e:
    st.error(f"Failed to create Timestream client: {e}")
    st.stop()

# SQL Query
QUERY = """
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END) AS
AttackSummary,
    MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
FROM "FleetManagementDB"."NetworkTable"
GROUP BY time
ORDER BY time DESC
```

```
"""

# Execute query
try:
    response = client.query(QueryString=QUERY)
    rows = response.get('Rows', [])
    columns = [col['Name'] for col in response.get('ColumnInfo', [])]

    if not rows or not columns:
        st.warning("No data returned from Timestream.")
        st.stop()

    # Parse rows
    data = [[col.get('ScalarValue', '') for col in row['Data']] for row in rows]
    df = pd.DataFrame(data, columns=columns)

except (BotoCoreError, ClientError) as e:
    st.error(f"Query failed: {e}")
    st.stop()

# Display table
st.subheader("🔍 Latest Network Logs")
st.dataframe(df)
```

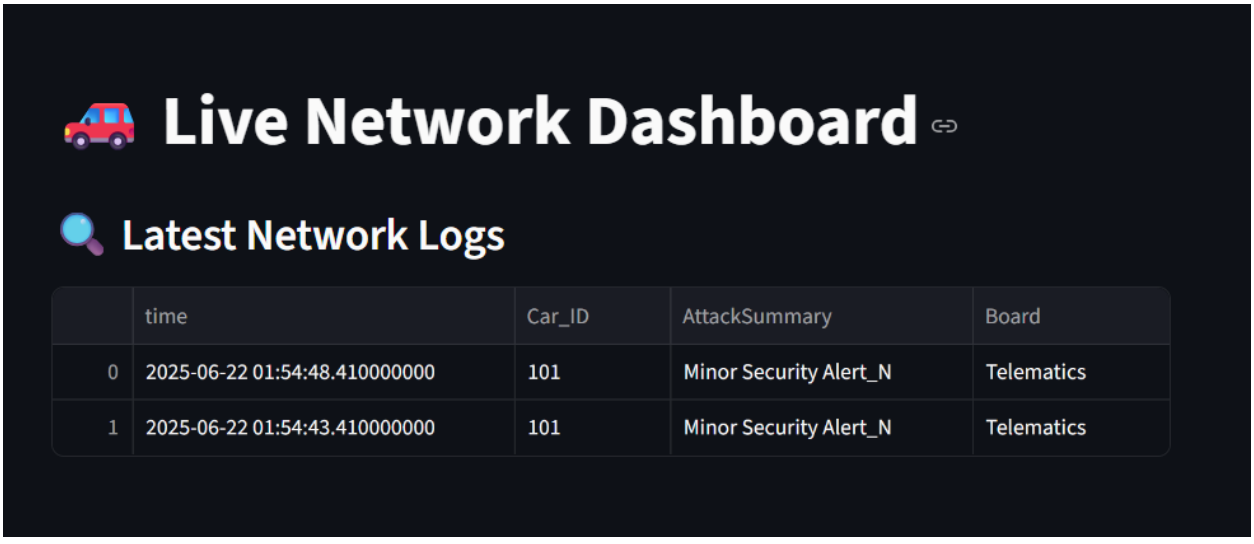

Figure 28 Final Results

References

[1] https://youtu.be/o5Wt0QS7Oco?si=qk6JGoD6jSdx1UYm

[2] https://docs.streamlit.io/

[3] https://drive.google.com/drive/u/1/folders/1GE_jTglje7QeSanW0_m_X94nfTSYz-1I?fbclid=IwAR0F2UZl12H90Yk42J0l3rFx-w11fEfEv6AkbURNaXNM81Kf1-2He8VKFao